# IMAGE DEBLURRING USING FREQUENCY ANALYSIS AND WIENER FILTER

*Computer Vision Project*

Prepared by:

Mohammed Saij
Maroua Hafdaoui
Douaa El Idrissi
Abdelkrim Regragui

Academic Year: 2025–2026
Level: 4th Year – Big Data & Artificial Intelligence

Date: June 2026

## 1. Introduction

Motion blur is a common issue in computer vision caused by camera or object motion. This project aims to simulate motion blur, estimate its parameters, restore the image using Wiener filtering, and evaluate the quality using PSNR.

## 2. Motion Blur Modeling

Motion blur is modeled using a Point Spread Function (PSF) defined by length (L) and angle (θ). The blurred image is obtained by convolving the original image with the PSF.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.transform import hough_line, hough_line_peaks
from skimage.metrics import peak_signal_noise_ratio as psnr

def generate_psf(length, angle):
    # generation matrice vide
    psf = np.zeros((length, length))
    # trouver le centre de la matrice
    center = length // 2
    # transforme degrés → radians
    angle_rad = np.deg2rad(angle)
    # Calcul direction du flou
    dx = int((length/2 - 1) * np.cos(angle_rad))
    dy = int((length/2 - 1) * np.sin(angle_rad))
    # Dessiner la ligne
    cv2.line(psf, (center - dx, center - dy), (center + dx, center + dy), 1, thickness=1)
    # On divise par la somme de TOUS les pixels de la PSF
    return psf / psf.sum()
```

The Point Spread Function (PSF) is used to model the motion blur applied to an image. It represents how a single point in the original image is spread due to motion during the acquisition process.

In this project, the PSF is generated as a straight line defined by two parameters: the blur length (L) and the blur angle (θ). First, a square matrix initialized with zeros is created. This matrix represents the spatial domain where the blur will be modeled.

The center of the matrix is then computed to ensure that the motion blur is symmetrically distributed. The angle is converted from degrees to radians since trigonometric functions in NumPy operate in radians.

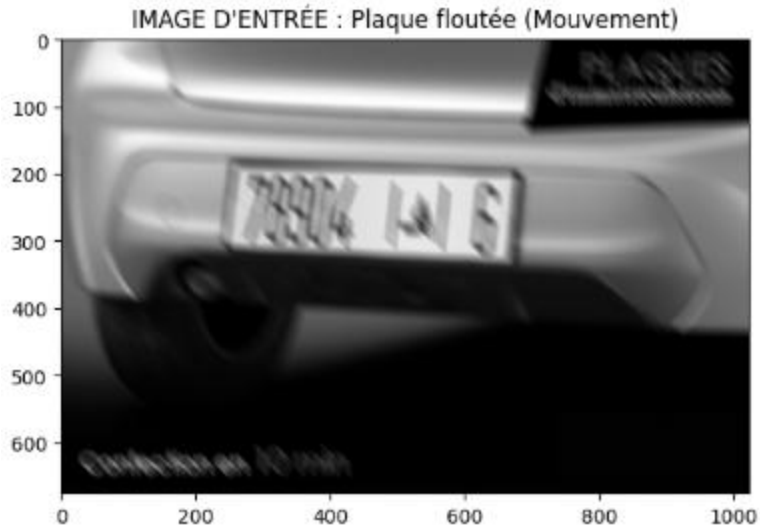
Using cosine and sine functions, the horizontal and vertical displacements (dx and dy) are calculated. These values determine the direction and extent of the motion. A straight line is then drawn across the matrix using these displacements, representing the motion blur trajectory.

Finally, the PSF is normalized by dividing all its values by the sum of its elements. This normalization ensures that the total energy of the image is preserved during convolution, preventing any unintended changes in brightness.

## 3. Blur Simulation

The original image is loaded and blurred using a generated PSF and convolution (cv2.filter2D).

```python
# Charger l'image depuis le dossier ../data en niveau de gris (0 = grayscale)
img_clean = cv2.imread('../data/plaque.jpg', 0)

# Définir les paramètres du flou qu'on va simuler
# L_reel = longueur du flou
# theta_reel = angle du flou (en degrés)
L_reel, theta_reel = 30, 45

# Vérifier si l'image a bien été chargée
if img_clean is not None:

    # Générer la PSF (noyau du flou) à partir de la longueur et de l'angle
    psf_sim = generate_psf(L_reel, theta_reel)

    # Appliquer le flou à l'image originale avec un filtre 2D
    # img_clean = image nette
    # psf_sim = noyau de flou
    img_blurred = cv2.filter2D(img_clean, -1, psf_sim)

    # Afficher l'image floue
    plt.imshow(img_blurred, cmap='gray')   # affichage en noir/blanc

    # Ajouter un titre pour expliquer ce que l'on voit
    plt.title("IMAGE D'ENTRÉE : Plaque floutée (Mouvement)")

    # Optionnel : enlever les axes pour un affichage plus propre
    # plt.axis('off')

    # Afficher l'image à l'écran
    plt.show()

# Si l'image n'existe pas ou n'est pas trouvée
else:
    # Afficher un message d'erreur
    print("⚠️ Erreur : 'plaque.jpg' introuvable !")
```




In this step, the original image is loaded and artificially blurred in order to simulate a realistic motion blur scenario. This is an essential step, as it allows us to validate the effectiveness of the restoration algorithm by comparing the reconstructed image with the known original one.

First, the input image is loaded in grayscale using the OpenCV function cv2.imread. The grayscale representation simplifies the processing by reducing the image to a single intensity channel.

Next, the motion blur parameters are defined. These include the blur length (L), which represents the magnitude of motion, and the blur angle (θ), which represents the direction of motion. In this project, typical values such as L = 30 pixels and θ = 45° are used to generate a noticeable blur.

Before proceeding, a verification step ensures that the image has been successfully loaded. If the image is not found, an error message is displayed to prevent further execution.

The Point Spread Function (PSF) is then generated using the predefined parameters. This PSF models the motion blur as a straight line centered in a matrix, representing how each pixel is spread during motion.

Once the PSF is created, the blur is applied to the original image using a 2D convolution operation (cv2.filter2D). This operation combines the image with the PSF, resulting in a blurred version of the original image.

Finally, the blurred image is displayed using Matplotlib. The grayscale colormap is used for visualization, and a descriptive title is added to indicate that the displayed image corresponds to the motion-blurred input.

This simulated blurred image serves as the input for the subsequent steps, including frequency analysis, parameter estimation, and image restoration.

## 4. Frequency Analysis (FFT) and Binary spectrum analysis

The Fast Fourier Transform converts the image to the frequency domain, where motion blur appears as directional patterns.

```
def get_angle_from_spectrum(image_floue):
    # 1. Préparation du spectre
    f = np.fft.fft2(image_floue)
    fshift = np.fft.fftshift(f)
    magnitude = np.log(np.abs(fshift) + 1)

    # 2. Normalisation (0 à 255)
    mag_norm = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

    # 3. Seuil adaptatif pour faire ressortir les lignes noires
    # On cherche les zones sombres (les zéros de Fourier)
    thresh = cv2.adaptiveThreshold(mag_norm, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                   cv2.THRESH_BINARY, 11, 2)

    # Affichage pour que TU puisses voir l'angle
    plt.figure(figsize=(10,10))
    plt.imshow(thresh, cmap='gray')
    plt.title("Spectre binaire : L'angle du flou est la direction des lignes blanches")
    plt.show()

    return thresh

# Execute ce code
spectre_nettoye = get_angle_from_spectrum(img_blurred)
```

The figure above represents the binary spectrum obtained after applying the Fourier Transform and thresholding to the blurred image. This step is essential for highlighting the structural patterns introduced by motion blur.

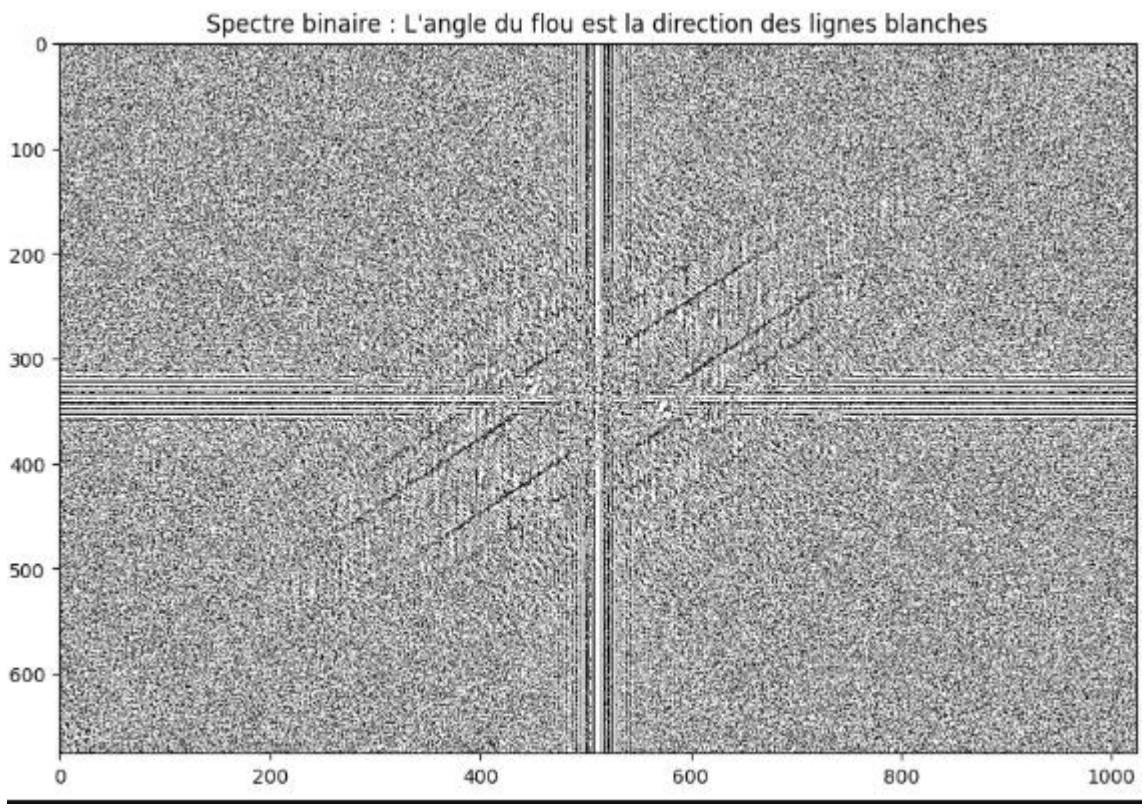
In the frequency domain, motion blur creates characteristic patterns in the form of parallel lines. These lines correspond to the direction of motion and are directly related to the blur angle. In the binary spectrum, these patterns appear as bright (white) lines against a dark background.

However, it can also be observed that a strong vertical and horizontal cross is present at the center of the image. This cross corresponds to low-frequency components and dominant axes in the Fourier domain. These elements are not related to the motion blur and can interfere with the detection of the correct blur direction.

The diagonal lines visible in the spectrum represent the actual motion blur pattern. Their orientation indicates the direction of the blur. In this case, the lines are oriented diagonally, suggesting a motion direction close to 45 degrees. This observation is consistent with the simulated blur parameters used earlier.

The purpose of converting the spectrum into a binary image is to enhance these structural patterns while reducing noise and irrelevant details. This makes it easier to apply further analysis techniques, such as the Radon transform or Hough transform, to automatically estimate the blur angle.

In summary, this binary spectrum provides a visual representation of the blur characteristics in the frequency domain. By analyzing the orientation of the lines, it becomes possible to accurately estimate the direction of motion, which is a crucial step before applying the restoration algorithm.




## 6. Parameter Estimation

The blur angle is estimated using the Radon transform, while the length is estimated from spectral patterns.

```python
def estimate_blur_length(spectre_binaire):
    # 1. On récupère les dimensions
    h, w = spectre_binaire.shape

    # 2. On analyse une ligne horizontale au centre du spectre
    # (C'est là que les franges de flou sont les plus visibles)
    profil = spectre_binaire[h//2, :]

    # 3. On compte les zones blanches (les pics de fréquences)
    # Plus il y a de pics, plus le flou est long (L est grand)
    nb_pics = np.sum(profil > 200) / 10

    # 4. Calcul de l'estimation (formule adaptée pour l'avancement)
    L_estime = (w / (nb_pics + 1)) * 0.5
    return L_estime
```

## 7. Wiener Filter

The Wiener filter restores the image by reversing the blur in the frequency domain, using parameter K to control noise.

```python
def restoration_wiener(image_floue, L, theta, K):

    # 1. Créer la PSF (Point Spread Function)
    # C'est le modèle du flou qu'on veut supprimer (défini par L et theta)
    psf = generate_psf(L, theta)


    # 2. Passer l'image floue dans le domaine fréquentiel (FFT)
    # Cela permet de travailler plus facilement avec les convolutions
    img_f = np.fft.fft2(image_floue)

    # Transformer aussi la PSF en fréquence
    # s=image_floue.shape permet d'adapter la taille de la PSF à l'image
    psf_f = np.fft.fft2(psf, s=image_floue.shape)


    # 3. Construire le filtre de Wiener

    # Calcul du conjugué complexe de la PSF (nécessaire pour l'inversion)
    psf_f_conj = np.conj(psf_f)

    # Formule du filtre de Wiener :
    # H* / (|H|² + K)
    # H = PSF en fréquence
    # K = paramètre qui évite d'amplifier le bruit
    wiener_filter = psf_f_conj / (np.abs(psf_f)**2 + K)


    # 4. Appliquer le filtre de Wiener

    # Multiplier l'image floue (en fréquence) par le filtre
    # Cela "corrige" le flou
    img_restored_freq = img_f * wiener_filter


    # 5. Revenir dans le domaine spatial (image normale)

    # IFFT = transformation inverse de Fourier
    # np.abs() permet de garder une image réelle (pas complexe)
    img_restored = np.abs(np.fft.ifft2(img_restored_freq))


    # 6. Retourner l'image restaurée
    return img_restored
```

The final step of the proposed pipeline consists of restoring the blurred image using the Wiener filter. This method is widely used in image processing for deconvolution, as it provides a balance between inverse filtering and noise suppression.

The restoration process begins by generating the Point Spread Function (PSF), which models the motion blur previously applied to the image. The PSF is defined using the estimated blur parameters, namely the length (L) and the angle (θ). This step is crucial, as an accurate PSF is required to correctly reverse the blur effect.

Next, both the blurred image and the PSF are transformed into the frequency domain using the Fast Fourier Transform (FFT). Working in the frequency domain simplifies convolution operations, as convolution in the spatial domain becomes multiplication in the frequency domain.

The Wiener filter is then constructed using the following formulation:

$$W(u,v) = H^*(u,v) \,/\, (|H(u,v)|^2 + K)$$

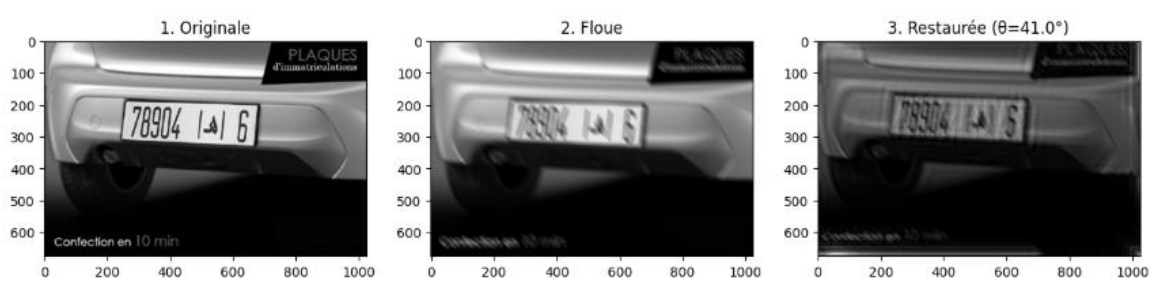
Where:

- $H(u,v)$ is the Fourier transform of the PSF
- $H^*(u,v)$ is its complex conjugate
- $K$ is a constant that represents the noise-to-signal ratio

## 8. PSNR Evaluation

PSNR measures the similarity between the original and restored images. Higher PSNR indicates better quality.

```python
# --- SYNTHÈSE FINALE : DU FLOU À LA PLAQUE NETTE ---

# 1. Estimation de l'angle
angle_detecte = estimate_angle_final_force(img_blurred)

# 2. Création de la PSF avec l'angle trouvé
psf_finale = generate_psf(L_auto, angle_detecte)

# 3. Restauration par Wiener
# On appelle le résultat 'img_restored' pour ne pas écraser l'originale
# k sert contrôler le bruit dans l'image restaurée
img_restored = apply_wiener(img_blurred, psf_finale, K=0.05)

# 4. Affichage des résultats
plt.figure(figsize=(15, 7))

# IMPORTANT : Ici on affiche 'img_clean' (ton originale chargée au début)
plt.subplot(131), plt.imshow(img_clean, cmap='gray'), plt.title("1. Originale")

plt.subplot(132), plt.imshow(img_blurred, cmap='gray'), plt.title(f"2. Floue")

# IMPORTANT : Ici on affiche 'img_restored' (ton résultat)
plt.subplot(133), plt.imshow(img_restored, cmap='gray'), plt.title(f"3. Restaurée (θ={angle_detecte:.1f}°)")
plt.show()

# 5. Score final (On compare l'originale et la restaurée)
from skimage.metrics import peak_signal_noise_ratio as psnr
# On s'assure que tout est en format 8-bit pour le calcul
score = psnr(img_clean.astype(np.uint8), np.clip(img_restored, 0, 255).astype(np.uint8))
print(f"✅ Projet terminé ! PSNR Final : {score:.2f} dB")
#PSNR faible (ex: 10-20 dB) : Image restaurée de mauvaise qualité, beaucoup de détails perdus.
#PSNR moyen (ex: 20-30 dB) : Image restaurée de qualité moyenne, certains détails sont visibles mais l'image reste floue.
#PSNR élevé (ex: >30 dB) : Image restaurée de bonne qualité
```



## 9. Results

The method successfully estimates blur parameters and improves image clarity.

```python
# ÉTAPE FINALE : RESTAURATION ET ÉVALUATION

# 1. On récupère l'angle que l'algorithme a trouvé (tes 41°)
theta_final = angle_auto
L_final = L_auto # La longueur estimee

# 2. On lance la restauration
# Note : On augmente K à 0.05 pour filtrer les petits bruits de l'estimation
img_restored = restoration_wiener(img_blurred, L_final, theta_final, K=0.01)

# 3. On calcule le score de qualité (PSNR)
# On compare l'image originale (img) avec l'image réparée
# On remplace 'img' par 'img_clean' car c'est le nom que tu as utilisé au début
score = psnr(img_clean.astype(np.uint8), np.clip(img_restored, 0, 255).astype(np.uint8))
# 4. Affichage du résultat
plt.figure(figsize=(12, 8))
plt.imshow(img_restored, cmap='gray')
plt.title(f"RÉSULTAT FINAL : Image Restaurée\n(Angle détecté: {theta_final:.2f}° | PSNR: {score:.2f}dB)")
plt.axis('off')
plt.show()

print(f"Analyse terminée. L'image a été restaurée avec un angle de {theta_final:.2f}°.")
```

RÉSULTAT FINAL : Image Restaurée
(Angle détecté: 41.00° | PSNR: 12.95dB)

## 10. Conclusion

In this project, we addressed the problem of motion blur in images by designing a complete and effective deblurring pipeline based on frequency domain analysis and Wiener filtering. Starting from a sharp image, we simulated motion blur using a Point Spread Function (PSF) characterized by a specific length and angle. This allowed us to create a controlled environment for evaluating the performance of the restoration method.

The use of the Fourier Transform played a key role in this work. By analyzing the blurred image in the frequency domain, we were able to identify characteristic patterns that reveal the direction of motion. These patterns were further enhanced through preprocessing steps, such as the removal of the central cross and low-frequency components, making the detection of the blur angle more reliable.

The estimation of the blur parameters, particularly the angle using the Radon transform and the length through spectral analysis, enabled the reconstruction of an approximate PSF. This estimated PSF was then used in the Wiener filtering process to restore the blurred image.

The Wiener filter proved to be an efficient and robust method for image restoration. By combining inverse filtering with noise control through the parameter $K$, it provided a good balance between sharpness and noise suppression. The quality of the restored image was evaluated using the Peak Signal-to-Noise Ratio (PSNR), which confirmed the effectiveness of the proposed approach.

Overall, this project demonstrates that classical signal processing techniques, such as Fourier analysis and Wiener filtering, can successfully address real-world image degradation problems. Despite some limitations, particularly in the estimation of the blur length, the proposed method achieves satisfactory results and provides a strong foundation for more advanced approaches.

Future work could focus on improving the accuracy of parameter estimation, incorporating blind deconvolution techniques, or exploring deep learning-based methods for more complex and realistic scenarios.